

DiffPF: Differentiable Particle Filtering with Generative Sampling via Conditional Diffusion Models

Ziyu Wan and Lin Zhao

Abstract—This paper proposes DiffPF, a *differentiable* particle filter that leverages *diffusion* models for state estimation in dynamic systems. Unlike conventional differentiable particle filters, which require importance weighting and typically rely on predefined or low-capacity proposal distributions, DiffPF learns a flexible posterior sampler by conditioning a diffusion model on predicted particles and the current observation. This enables accurate, equally-weighted sampling from complex, high-dimensional, and multimodal filtering distributions. We evaluate DiffPF across a range of scenarios, including both unimodal and highly multimodal distributions, and test it on simulated as well as real-world tasks, where it consistently outperforms existing filtering baselines. In particular, DiffPF achieves a 90.3% improvement in estimation accuracy on a highly multimodal global localization benchmark, and a nearly 50% improvement on the real-world robotic manipulation benchmark, compared to state-of-the-art differentiable filters. To the best of our knowledge, DiffPF is the first method to integrate conditional diffusion models into particle filtering, enabling high-quality posterior sampling that produces more informative particles and significantly improves state estimation. The code is available at <https://github.com/ZiyuNUS/DiffPF>.

Index Terms—Differentiable particle filtering, diffusion models, Bayesian state estimation, visual odometry, generative models.

I. INTRODUCTION

ESTIMATING the hidden state of a dynamic system from partial and noisy observations is a core challenge across a wide range of applications. Kalman filters and their extensions [1] have long served as standard tools within the Bayesian filtering framework. However, their reliance on linearity and Gaussian noise assumptions severely limits their applicability to real-world systems that are often nonlinear, high-dimensional and exhibit multimodal behaviors.

An alternative line of work, particle filters, avoids these restrictive assumptions by representing the posterior distribution with a set of weighted samples instead of relying on closed-form expressions. Bootstrap particle filter [2], one of the most commonly used algorithms, draws particles from the transition model and assigns weights based on observation likelihoods. Rao-Blackwellized particle filter [3] further improves efficiency by analytically marginalizing part of the

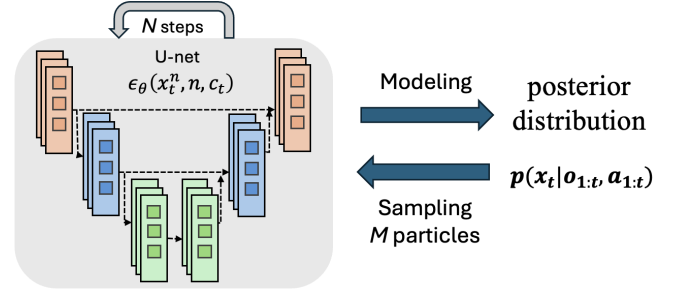


Fig. 1: A conditional diffusion model parameterized by a U-Net is used to model the filtering posterior distribution and generate equally weighted particles samples via iterative denoising.

latent state using parametric inference, thereby reducing sampling variance. However, these methods still rely on manually designed or limited-capacity proposals and are prone to sample degeneracy, particularly in high-dimensional or perceptually complex domains.

Recent developments in deep learning have introduced data-driven alternatives to traditional state estimation methods. Deep state-space models (DSSMs) [4] provide a unified framework capable of capturing nonlinear latent dynamics and processing high-dimensional inputs, making them particularly suited for complex real-world environments. Differentiable Particle Filters (DPFs), a class of methods that embed sample-based filtering into DSSMs [5], [6], enable end-to-end learning of recursive Bayesian inference over learned dynamics and observation models. However, most DPFs still inherit several limitations from classical particle filtering. The proposal distributions used in DPFs are often either hand-crafted or limited in capacity, making it difficult to match the true filtering posterior [7]. Furthermore, they rely on importance sampling, which necessitates explicit weighting of particles—a process that becomes increasingly unreliable in high-dimensional or highly multimodal settings due to particle degeneracy.

To overcome these challenges, we employ diffusion models as powerful implicit samplers, capable of capturing rich and multimodal posterior distributions. This allows for a more expressive and learnable formulation of the state update step within the filtering process. Specifically, as illustrated in Fig. 1, DiffPF employs a diffusion model for state update, conditioned on both the predicted particles and the current observation to generate refined posterior samples. DiffPF departs from particle filtering by eliminating the reliance on importance sampling and explicit proposal distributions. Instead, it leverages a diffusion model to sample directly from the filtering posterior, enabling accurate inference without the need for particle weighting. This sampling-based yet weight-free approach naturally avoids particle degeneracy and, consequently, elimi-

Manuscript received: June 29, 2025; Revised: October 17, Accepted: January 2, 2026. This paper was recommended for publication by Editor Aleksandra Faust upon evaluation of the reviewers' comments. This work was supported by the Singapore Ministry of Education Tier 2 Academic Research Funds (T2EP20123-0037, T2EP20224-0035). (Corresponding author: Lin Zhao)

The authors are with the Department of Electrical and Computer Engineering, National University of Singapore, Singapore. (e-mail: wzyu@u.nus.edu, elezhli@nus.edu.sg)

Digital Object Identifier (DOI): see top of this page.

nates the need for the non-differentiable resampling step, while supporting multimodal and high-dimensional distributions.

Our main contributions are summarized as follows:

- We introduce a differentiable filtering framework that employs diffusion models for latent states estimation. To the best of our knowledge, this is the first application of diffusion models in differentiable particle filtering.
- Our method enables high-quality, weight-free inference by implicitly modeling and sampling from the posterior distribution. This eliminates particle degeneracy and removes the need for manually designed proposals, while supporting high-dimensional, multimodal posteriors in a fully differentiable and end-to-end trainable paradigm.
- We evaluate DiffPF across a range of synthetic and real-world scenarios, covering both unimodal and multimodal distributions. Across these benchmarks, our approach achieves significant improvements over existing state-of-the-art differentiable filtering methods.

II. RELATED WORKS

A. Particle filter

Particle filters (PFs) are a class of sequential Monte Carlo methods widely used for recursive state estimation in nonlinear and non-Gaussian systems [8]. Classical variants, such as the Bootstrap Filter [2], follow the Sampling-Importance-Resampling framework [9], which approximates the filtering distribution with weighted particles. While theoretically general, traditional PFs often suffer from particle degeneracy—where most weights collapse to near zero—and from the difficulty of designing effective proposal distributions, especially in high-dimensional or perceptually complex environments. Various extensions [9] have been developed to mitigate these issues by improving sampling efficiency, yet they typically remain limited by the same challenges of weight collapse and proposal design in complex state spaces.

B. Differentiable Particle Filters

The aforementioned traditional methods often assume known system dynamics or observation models, which often do not hold in real-world scenarios [10]. Differentiable particle filters [5]–[7], [11]–[15] address this by integrating neural networks into the filtering process, enabling end-to-end learning from data.

Most DPFs model transition dynamics by mapping previous states to the next latent state using either fully connected networks [6], [7], recurrent architectures such as LSTMs and GRUs [12], [15], or fixed parametric forms with learnable noise parameters [14]. Observation models are typically implemented as neural networks that either output the parameters of a predefined distribution (e.g., the mean and variance of a Gaussian) [15] or produce unnormalized scores interpreted as pseudo-likelihoods [11]. While effective in simple scenarios, these designs are often restricted to specific distribution families and struggle to represent complex or multimodal posteriors, limiting the expressiveness of the resulting filtering distributions and motivating the need for a more flexible and general framework.

The proposal distribution, which guides particle sampling, is typically handcrafted [16], [17] or implemented as a neural network conditioned on the current observation and possibly past states. In many DPFs, the transition model is reused as the proposal, and particles are often generated deterministically via regression rather than sampling, which leads to particle degeneracy [18], reduces particle quality, and constrains the representational capacity of the filtering distribution. Although [10] proposes using normalizing flows to sample from the proposal distribution, the method remains confined to the importance sampling framework and suffers from accumulated estimation errors due to sampling from multiple distributions simultaneously. To enable gradient-based learning, DPFs must address the non-differentiability of traditional resampling operations. Several methods replace hard resampling with differentiable approximations [19], such as soft-resampling [7] and entropy-regularized optimal transport solvers [11]. While these approximations allow gradients to flow through resampling steps, they often introduce additional computational complexity or require task-specific tuning, and particle degeneracy can still persist.

C. Diffusion Models for Probabilistic Modeling

Generative modeling has gained increasing attention across diverse domains [20], [21]. Recent advances in generative modeling have highlighted diffusion models as effective tools for capturing intricate, multimodal distributions across a range of data types. These models operate by progressively denoising latent variables [22], [23], making them particularly adept at representing uncertainty and complex data structures.

In the domain of robotics, such approaches as Diffusion Policy [24] and NoMaD [25] have demonstrated the effectiveness of conditional diffusion models in generating action sequences by capturing the inherently multi-modal nature of action distributions [26]. Meanwhile, diffusion models have gained significant attention for their effectiveness in trajectory generation [27], [28]. Building on the success of these tasks, we explore their potential in state estimation. Existing attempts include introducing diffusion models into Kalman filters [29]. We further develop a novel sample-based framework that integrates diffusion models into differentiable particle filtering, and propose end-to-end efficient training algorithms to facilitate the filter design.

III. METHOD

In sequential inference, a Bayesian filter aims to recursively estimate the current state $\mathbf{x}_t$ via the posterior distribution $\text{post}(\mathbf{x}_t) := p(\mathbf{x}_t \mid \mathbf{o}_{1:t}, \mathbf{a}_{1:t})$. Under the Markov assumption, all relevant information from past observations $\mathbf{o}_{1:t}$ and actions $\mathbf{a}_{1:t}$ is captured by the current posterior $\text{post}(\mathbf{x}_t)$. By exploiting the conditional independence of observations and actions given the latent state, the inference proceeds recursively in two steps—prediction and update step [6], [30]. As illustrated in Figure 2, our method follows this two-step Bayesian filtering procedure and represents the posterior $\text{post}(\mathbf{x}_t)$ using a set of equally weighted particles $\chi_t := \{\mathbf{x}_t^{(i)}\}_{i=1}^N$. The remainder of this section details each component of the proposed framework.

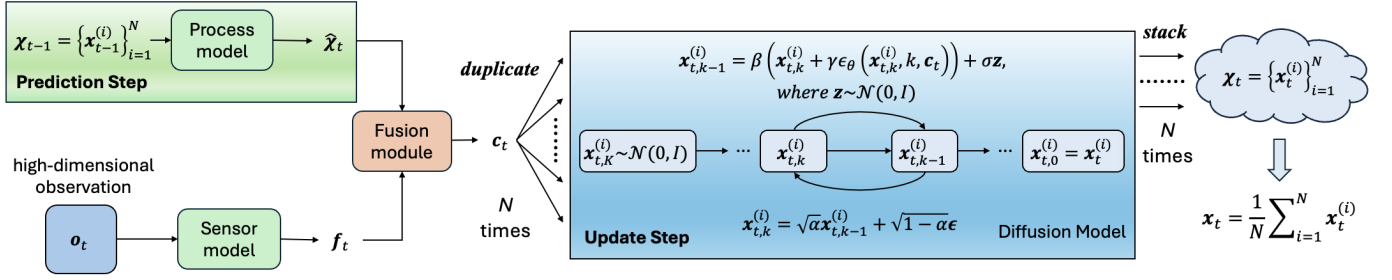


Fig. 2: At time t , the estimated particles from time $t-1$ are propagated through the process model to obtain a prior distribution over the current state. Simultaneously, the observation $\mathbf{o}_t$ is encoded into a feature representation. These two sources of information jointly condition a diffusion model, which iteratively refines a set of noisy latent samples to generate equally weighted particles that approximate the filtering posterior. Unlike DnD Filter [29], which uses a diffusion model in a Kalman filter-like manner to fuse prior and observation into a single trajectory, DiffPF maintains a full set of particles to explicitly represent the posterior distribution.

A. Prediction and Perception Modeling

Given a set of N particles $\{\mathbf{x}_{t-1}^{(i)}\}_{i=1}^N$ representing the posterior at time $t-1$, we apply a process model to obtain predicted particles $\hat{\mathbf{x}}_t := \{\hat{\mathbf{x}}_t^{(i)}\}_{i=1}^N$ for the current step. Specifically, each particle is propagated through a learnable or given process model f_{dyn} as:

$$\hat{\mathbf{x}}_t^{(i)} = f_{\text{dyn}}(\mathbf{x}_{t-1}^{(i)}, \mathbf{a}_t), \quad (1)$$

where $\mathbf{a}_t$ is the action at time t . Through this step, the relevant information from the action sequence $\mathbf{a}_{1:t}$ and past observations $\mathbf{o}_{1:t-1}$ is encoded into the predicted particle set $\{\hat{\mathbf{x}}_t^{(i)}\}_{i=1}^N$. The process model f_{dyn} can be defined using either parametric forms based on system knowledge, or learned from data using neural networks such as Multi-Layer Perceptrons (MLPs), stochastic models [31], or generative approaches like normalizing flows [32].

In parallel, the current observation $\mathbf{o}_t$ is processed by a neural sensor model g_{obs} to extract observation features:

$$\mathbf{f}_t = g_{\text{obs}}(\mathbf{o}_t). \quad (2)$$

This feature representation encodes perceptual information relevant to the latent state.

Unlike conventional differentiable filters that rely on predefined distribution families or manually specified noise models to represent the dynamics and observation processes, our approach imposes fewer assumptions and offers better generalization across scenarios.

To combine the information from the prior and observation $\mathbf{o}_t$, we incorporate a learnable fusion mechanism that aggregates the predicted particles and perceptual embeddings:

$$\mathbf{c}_t = \text{Fusion}(\mathbf{f}_t, \{\hat{\mathbf{x}}_t^{(i)}\}_{i=1}^N), \quad (3)$$

where $\mathbf{c}_t$ aggregates temporal information from $\mathbf{o}_{1:t}$ and $\mathbf{a}_{1:t}$ via the process and sensor models, and serves as the conditioning input for the subsequent diffusion process.

B. Update Step

Update step refines the prior belief over the current state by incorporating the latest observation. In conventional DPFs, the update is typically implemented by assigning importance weights to particles based on an observation model, followed by resampling to mitigate particle degeneracy. However, these

procedures often require carefully designed likelihood functions and involve non-differentiable operations. In contrast, our method performs the update implicitly through a conditional diffusion model. Specifically, we make an assumption that

$$\text{post}(\mathbf{x}_t) \approx p(\mathbf{x}_t | \mathbf{c}_t). \quad (4)$$

This assumption is justified because $\mathbf{c}_t$ captures all temporal information from $\mathbf{o}_{1:t}$ and $\mathbf{a}_{1:t}$. Instead of sampling directly from the intractable posterior distribution $p(\mathbf{x}_t | \mathbf{o}_{1:t}, \mathbf{a}_{1:t})$, we choose to sample from the approximated distribution $p(\mathbf{x}_t | \mathbf{c}_t)$. This yields a set of particles $\{\mathbf{x}_t^{(i)}\}_{i=1}^N$, from which we can obtain:

$$\text{post}(\mathbf{x}_t) \approx \frac{1}{N} \sum_{i=1}^N \delta(\mathbf{x}_t - \mathbf{x}_t^{(i)}), \quad (5)$$

where $\delta(\cdot)$ denotes the Dirac function. This indicates that all sampled particles share equal weights, thereby eliminating the need for weighting or resampling.

Diffusion models have been shown to approximate arbitrarily complex probability distributions by learning time-dependent score functions that guide the reversal of a noise-perturbed process [33]. In our case, a conditional diffusion model is trained to approximate $p(\mathbf{x}_t | \mathbf{c}_t)$ and generate particle samples. Specifically, we instantiate a reverse diffusion process with a Denoising Diffusion Probabilistic Model (DDPM), which iteratively refines a random Gaussian sample toward the target posterior distribution. We begin by drawing an initial Gaussian sample:

$$\mathbf{x}_{t,K}^{(i)} \sim \mathcal{N}(0, I), \quad (6)$$

where K is the total number of reverse diffusion steps. At each step k , the model predicts the noise component $\epsilon_\theta(\mathbf{x}_{t,k}^{(i)}, k, \mathbf{c}_t)$, which subsequently serves to infer a cleaner approximation of the sample at timestep $k-1$:

$$\begin{aligned} \mathbf{x}_{t,k-1}^{(i)} &= \frac{1}{\sqrt{\alpha}} \left(\mathbf{x}_{t,k}^{(i)} - \frac{1-\alpha}{\sqrt{1-\bar{\alpha}}} \epsilon_\theta(\mathbf{x}_{t,k}^{(i)}, k, \mathbf{c}_t) \right) + \sigma \mathbf{z}, \\ &:= \beta(\mathbf{x}_{t,k}^{(i)} + \gamma \epsilon_\theta(\mathbf{x}_{t,k}^{(i)}, k, \mathbf{c}_t)) + \sigma \mathbf{z}. \end{aligned} \quad (7)$$

Here $\mathbf{z} \sim \mathcal{N}(0, I)$, and $\mathbf{x}_{t,k}^{(i)}$ refers to the i -th corrupted sample at diffusion step k . α , $\bar{\alpha}$, σ are fixed noise schedule parameters as defined in [34], while β and γ are scaling terms introduced for simplicity. This stochastic refinement continues recursively

until $k = 0$, yielding the final particle sample $\mathbf{x}_{t,0}^{(i)} = \mathbf{x}_t^{(i)}$. This reverse diffusion process is repeated N times to produce the final particle set $\chi_t = \{\mathbf{x}_t^{(i)}\}_{i=1}^N$. As all particles are equally weighted, the final estimate is given by the average over the particle set

$$\mathbf{x}_t = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_t^{(i)}. \quad (8)$$

By leveraging a diffusion model for the update step, our framework effectively approximates complex and multimodal posterior distributions and samples high-quality samples while preserving full differentiability. Moreover, it removes the need for handcrafted proposals and circumvents the weighting and resampling procedures required in importance sampling frameworks, thereby mitigating inefficiencies and avoiding particle degeneracy.

C. End-to-End Training

The diffusion model is trained following the standard DDPM formulation [23]. At each training iteration, a noisy version of the ground-truth state $\mathbf{x}_{t,k}^*$ is generated by adding Gaussian noise to the clean state $\mathbf{x}_{t,0}^*$, based on a randomly selected diffusion timestep k . The denoising network ϵ_θ , implemented as a U-Net (see Figure 1), is trained to predict the added noise ϵ conditioned on the diffusion timestep k and the fusion vector $\mathbf{c}_t$. The training objective minimizes the mean squared error between the predicted and true noise:

$$\mathcal{L} = \mathbb{E}_{\mathbf{x}_{t,0}^*, k, \mathbf{c}_t, \epsilon} [\|\epsilon - \epsilon_\theta(\mathbf{x}_{t,k}^*, k, \mathbf{c}_t)\|^2]. \quad (9)$$

All components of the system—including the process model, observation encoder, and fusion module—are trained jointly with the diffusion model in an end-to-end manner to ensure consistent optimization across the entire framework.

IV. EXPERIMENTS

We evaluate DiffPF across four challenging state estimation tasks: (1) a synthetic image-based object tracking task; (2) a simulated global localization task with pronounced multimodality; (3) the real-world KITTI visual odometry benchmark and (4) a robotic manipulation task.

We compare DiffPF against a comprehensive set of baselines in the following experiments, including the Deep State-Space Model (Deep SSM) [4], Particle Filter Recurrent Neural Networks (PFRNNs) [15], Particle Filter Networks (PFNNs) [7], Normalizing Flow-based Differentiable Particle Filters (NF-DPFs) [10], Autoencoder Sequential Monte Carlo (AESMC) and its bootstrap variant (AESMC-Bootstrap) [16], [17], as well as other differentiable filtering and smoothing methods [14], [35]–[37]. All DPF-based baselines are evaluated with 100 particles, while DiffPF uses 10 particles unless otherwise specified. The noise estimator in the diffusion model is implemented as a 3-layer U-Net in all experiments and the number of diffusion steps is set to 10 unless otherwise specified. DiffPF was trained and evaluated on a workstation equipped with an NVIDIA GeForce RTX 4080 GPU and an Intel i7-14700KF CPU. Training uses the AdamW optimizer with an initial learning rate of 1×10^{-4} and a cosine annealing

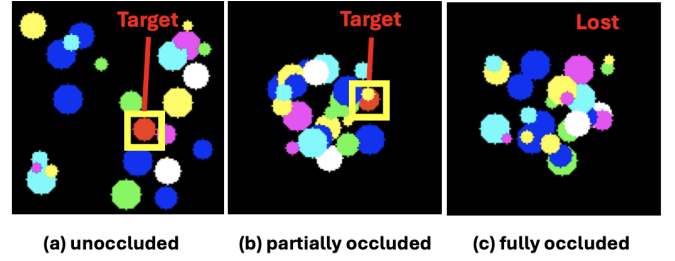


Fig. 3: A sequence of three frames with the target disk indicated by a yellow box. The frames illustrate three different levels of occlusion caused by interfering disks.

schedule. We train for 1000 epochs with warmup enabled for the first 4 epochs. For the disk tracking task we use a batch size of 50, while for global localization the batch size is 100. For KITTI visual odometry and robotic manipulation task, due to the fold-based training protocol, the batch size varies across folds. Full configurations are provided in our released code.

A. Vision-based Disk Tracking

Overview: The objective is to estimate the motion path of a specific disk within a crowded two-dimensional environment where numerous distractor disks move simultaneously. The position of the target disk is treated as the latent state. The main challenge of this task lies in the varying degrees of occlusion that occur during the target’s movement, as shown in Fig. 3, which introduces dynamic uncertainty into the observations. For filtering methods, it is essential to effectively leverage both the temporal information derived from prediction and the information extracted from the current observations.

Data: We follow exactly the same dataset and simulation setup as in [10]. The dataset consists of 500 training and 50 testing sequences, each containing 50 RGB frames of size 128×128 . Each frame includes one target disk and 25 distractor disks, initialized uniformly at random. The target red disk has a fixed radius of 7 pixels, while distractors vary in size and color. The disk dynamics follow the same formulation as in [14].

Implementation: At time step t , we use the given velocity $\mathbf{v}_t^*$ as input to generate a prior estimate of $\mathbf{x}_{t-1}^{(i)}$. The following formulation serves as our process model:

$$\hat{\mathbf{x}}_t^{(i)} = \mathbf{x}_{t-1}^{(i)} + \mathbf{v}_t^*. \quad (10)$$

Due to the imperfection of the process model, observation $\mathbf{o}_t$ is used to correct the prior estimate. We adopt the network architecture from [10] as our sensor model. To integrate $\{\hat{\mathbf{x}}_t^{(i)}\}_{i=1}^N$ with $\mathbf{o}_t$, we first convert the $\{\hat{\mathbf{x}}_t^{(i)}\}_{i=1}^N$ into a heatmap. The observation $\mathbf{o}_t$ is encoded by the sensor model, while the heatmap is processed by a separate network with the same architecture. The outputs of both networks are then concatenated to form the feature representation $\mathbf{c}_t$, which are used as conditions for the diffusion model.

Results: We evaluate our proposed DiffPF across three dimensions:

1) **Benchmarking.** We compare the DiffPF with several baselines, including DeepSSM, AESMC-Bootstrap, AESMC, PFRNN, PFNet, and NF-DPF, all of which adopt the same process and observation models where applicable.

TABLE I: Experimental results for ablation analysis and baseline comparisons. Mean and standard deviation are computed from 5 independent runs with varying random seeds.

Setting	Model	MSE (pixels)
Prior Works	Deep SSM	5.91 ± 1.20
	AESMC Bootstrap	6.35 ± 1.15
	AESMC	5.85 ± 1.34
	PFRNN	6.12 ± 1.23
	PFNet	5.34 ± 1.27
	NF-DPF	3.62 ± 0.98
Diffusion Step	DiffPF (5s)	1.50 ± 0.02
	DiffPF (10s)	1.37 ± 0.03
Priors Accuracy	DiffPF (no pred)	5.23 ± 0.18
	DiffPF ($\sigma = 2$)	1.44 ± 0.01
	DiffPF ($\sigma = 5$)	1.96 ± 0.02
	DiffPF ($\sigma = 10$)	3.37 ± 0.12

TABLE II: Inference frequency for different numbers of diffusion steps. Means are computed from 5 independent runs with varying random seeds.

Diffusion Step	K = 5	K = 10	K = 20	K = 50	K = 100
Infer. Freq. (Hz)	59.0	39.2	21.1	9.9	5.26

2) **Ablation analysis.** We further investigate the design of our method through controlled experiments, including:

- *Number of diffusion steps.* We compare two variants, **DiffPF(5s)** and **DiffPF(10s)**, which use 5 and 10 diffusion steps respectively;
- *Effect of the process model prior.* We perform an ablation study by injecting different levels of noise into the process model Eq. 10. The variants are denoted as **DiffPF(σ)**, where σ controls the standard deviation of the noise added to the dynamics. We also test the variant that removes the prior, denoted as **DiffPF(no pred)**.

3) **Inference efficiency.** We assess the runtime of the DiffPF under different diffusion step settings.

All methods are evaluated using the mean squared error (MSE) between the predicted and ground-truth trajectories of the target object. Results are summarized in Table I and Table II.

Comparison: As shown in Table I and Table II, DiffPF consistently outperforms existing differentiable particle filters in terms of both accuracy and stability while using significantly fewer particles, achieving a 62.2% improvement over NF-DPF. Increasing the number of diffusion steps further improves estimation accuracy, while inference remains real-time in most settings with a modest GPU memory footprint of 224.1 MB. Incorporating the predicted state from the process model yields a 73.8% reduction in MSE and DiffPF maintains strong robustness under prior perturbations.

B. Global Localization

Overview: The task is to estimate the robot’s pose based on observed images and odometry data, within three maze environments simulated in DeepMind Lab [38]. The main challenge of this task lies in the highly multimodal nature of the observations: since we adopt the same experimental setup as in [6], unique textures have been deliberately removed

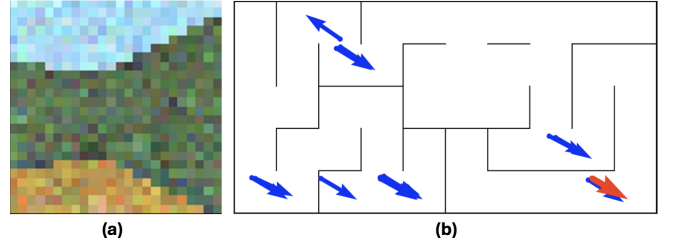


Fig. 4: (a) An example observation captured by the robot. (b) The red arrow indicates the true pose, while the blue arrows mark alternative poses in the map that yield similar observations, illustrating the multimodal nature of the observation-to-state mapping.

from the environment, causing many locations in the maze to produce nearly identical images. An example of the observation image is shown in Fig 4, along with the corresponding candidate poses on the map.

Data: Consistent with [10], the dataset is divided into 900 training sequences and 100 test sequences, with each sequence containing 100 time steps. The ground truth state is defined as the robot’s position and orientation, denoted as $\mathbf{x}_t^* := (p_{t,x}^*, p_{t,y}^*, \theta_t^*)$, and the action $\mathbf{a}_t^* = (u_t^*, v_t^*, \Delta\theta_t^*)$ corresponds to the robot’s velocity in its local frame. The original 32×32 RGB observations are randomly cropped to 24×24 and augmented with Gaussian noise ($\sigma = 20$) to introduce visual uncertainty.

Implementation: At time step t , we use the given input $\mathbf{a}_t^*$ to generate a prior estimate of $\mathbf{x}_t^{(i)} = (p_{t,x}^{(i)}, p_{t,y}^{(i)}, \theta_t^{(i)})$. We form the process model as:

$$\hat{\mathbf{x}}_t^{(i)} = \begin{bmatrix} p_{t,x}^{(i)} + u_t^* \cos(\theta_t^{(i)}) + v_t^* \sin(\theta_t^{(i)}) \\ p_{t,y}^{(i)} + u_t^* \sin(\theta_t^{(i)}) - v_t^* \cos(\theta_t^{(i)}) \\ \theta_t + \Delta\theta_t^* \end{bmatrix} + \boldsymbol{\omega}_t, \quad (11)$$

where $\boldsymbol{\omega}_t$ denotes zero-mean Gaussian noise with covariance $\boldsymbol{\Sigma} = \text{diag}(10^2, 10^2, 0.1^2)$. We adopt the same encoder from [10] as our sensor model. To integrate $\{\hat{\mathbf{x}}_t^{(i)}\}_{i=1}^N$ with $\mathbf{o}_t$, We encode the observation $\mathbf{o}_t$ using the sensor model and flatten the particle set $\{\hat{\mathbf{x}}_t^{(i)}\}_{i=1}^N$ into a vector. The two are concatenated to form the conditional input $\mathbf{c}_t$ for the diffusion model. The model is first pretrained using only observations, and then finetuned in a second training phase with particle-based priors integrated.

Results: DiffPF is compared with the same set of baselines as in the previous benchmark under a shared setup to assess overall performance. To further analyze the framework, we conduct several ablation studies. First, to evaluate the contribution of the process model prior, we include a variant trained without it, denoted as DiffPF (no pred), and additional variants where different levels of noise are injected into the prior (DiffPF(Σ)). Second, we investigate particle efficiency by varying the number of particles and measuring both inference frequency and memory footprint across tasks. All methods are evaluated using the root mean squared error (RMSE) between the predicted and ground-truth positions of the robot at the last step. The results are summarized in Table III, IV and V.

Comparison: As shown in Table III, DiffPF achieves superior performance over existing differentiable particle filters in terms of both accuracy and consistency, while operating with

TABLE III: RMSE between the predicted and ground-truth positions of the robot at the last step $t = 100$. Mean and standard deviation are computed from 5 independent runs with varying random seeds.

Method	Maze 1	Maze 2	Maze 3
Deep SSM	61.0 $\pm$ 10.8	114.0 $\pm$ 8.8	203.0 $\pm$ 10.1
AESMC Bootstrap	56.5 $\pm$ 11.5	115.6 $\pm$ 6.8	220.6 $\pm$ 11.1
AESMC	52.1 $\pm$ 7.5	109.2 $\pm$ 11.7	201.3 $\pm$ 14.7
PFNet	51.4 $\pm$ 8.7	120.3 $\pm$ 8.7	212.1 $\pm$ 15.3
PFRNN	54.1 $\pm$ 8.9	125.1 $\pm$ 8.2	210.5 $\pm$ 10.8
NF-DPF	46.1 $\pm$ 6.9	103.2 $\pm$ 10.8	182.2 $\pm$ 19.9
DiffPF	6.1 $\pm$ 0.5	7.6 $\pm$ 0.2	15.3 $\pm$ 1.2
DiffPF(2 Σ)	8.7 $\pm$ 0.3	11.7 $\pm$ 0.5	17.8 $\pm$ 1.3
DiffPF(5 Σ)	18.7 $\pm$ 1.2	25.2 $\pm$ 2.5	28.4 $\pm$ 2.4
DiffPF(10 Σ)	106.4 $\pm$ 16.1	61.7 $\pm$ 9.2	67.2 $\pm$ 17.4
DiffPF (no pred)	137.0 $\pm$ 3.8	213.0 $\pm$ 5.8	414.6 $\pm$ 12.3

TABLE IV: Effect of particle number N on RMSE at the last step $t = 100$, averaged over 5 runs.

Particle Number	Maze 1	Maze 2	Maze 3
$N = 1$	19.3 $\pm$ 14.2	37.3 $\pm$ 27.6	131.3 $\pm$ 53.0
$N = 5$	13.2 $\pm$ 0.9	11.1 $\pm$ 2.5	30.9 $\pm$ 14.2
$N = 10$	6.1 $\pm$ 0.5	7.6 $\pm$ 0.2	15.3 $\pm$ 1.2
$N = 20$	5.4 $\pm$ 0.2	7.6 $\pm$ 0.3	25.7 $\pm$ 2.7
$N = 40$	4.9 $\pm$ 0.1	8.2 $\pm$ 0.1	20.1 $\pm$ 6.8
$N = 80$	4.3 $\pm$ 0.2	7.7 $\pm$ 0.1	17.2 $\pm$ 0.4

TABLE V: Inference frequency and memory footprint for different numbers of particles. Means are computed from 5 independent runs with varying random seeds.

Particle number	$N = 5$	$N = 10$	$N = 20$	$N = 40$	$N = 80$
Infer. Freq. (Hz)	54.1	52.6	45.5	27.4	14.3
Memory (MB)	188.3	194.2	205.5	230.0	276.6

substantially fewer particles. Compared to NF-DPF—which also employs generative modeling—DiffPF achieves improvements of 86.8%, 92.6%, and 91.6% in Maze 1, Maze 2, and Maze 3, respectively. Furthermore, the notably lower variance observed across multiple runs underscores its robustness and reliability.

As shown in Fig. 5, even in the most complex and highly multimodal Maze 3 environment, DiffPF is able to rapidly converge to a pose close to the ground truth during the early stages, while other methods may require 50 or more steps to achieve similar accuracy. Incorporating the process model prior yields RMSE reductions of 95.5%, 96.4%, and 96.3% in the three maze environments, indicating that introducing the prior effectively reduces the multimodal ambiguity in the observation-to-state mapping. Furthermore, DiffPF demonstrates strong robustness under moderate perturbations of the prior, exhibiting only gradual performance degradation. However, when the noise level becomes extremely large (e.g., 10 Σ), as shown in Fig. 6, prior inaccuracies can occasionally lead to sample collapse, manifested as convergence to incorrect trajectories. This phenomenon likely stems from multiple factors, including limited coverage of such extreme perturbations in the training data, insufficient convergence due

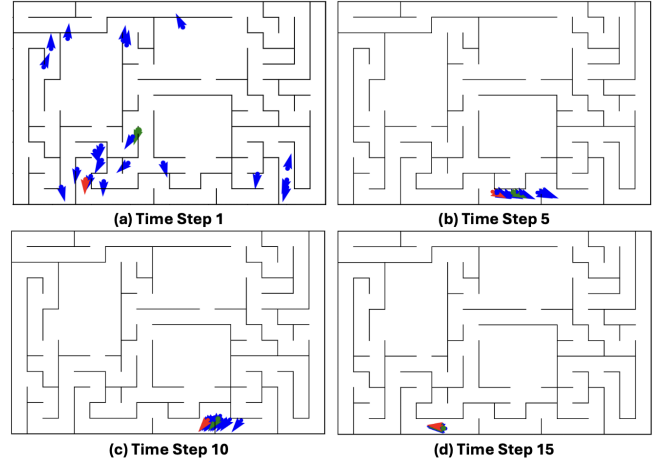


Fig. 5: Estimated poses at selected time steps from a sequence using DiffPF. Red arrows indicate the ground truth, green arrow represent the estimated pose, and blue arrows represent the particle set.

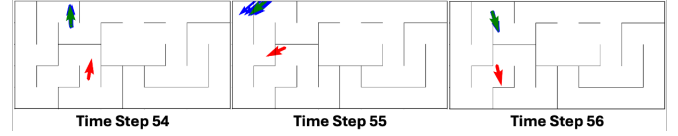


Fig. 6: An example where the proposed DiffPF collapses under highly noisy priors. The arrow notation is the same as in Fig. 5.

to suboptimal training or optimization, and constrained model capacity under highly perturbed dynamics. Nevertheless, even under this challenging setting, DiffPF still outperforms almost all baseline methods.

As for the effect of particle number, Table IV shows that increasing N generally improves estimation accuracy and stability. Notably, DiffPF already achieves low RMSE with as few as $N = 10$ particles, and further gains beyond $N = 40$ are relatively modest. This demonstrates the particle efficiency of our approach compared to traditional filters, which require more particles (100 particles) yet still fail to reach similar accuracy. In terms of scalability, inference frequency decreases and memory usage increases approximately linearly with N . Nevertheless, even with $N = 80$, DiffPF sustains real-time inference rates with moderate memory consumption, confirming its practicality while delivering strong accuracy with relatively few particles.

C. KITTI Visual Odometry

Overview: We evaluate our method on the KITTI Visual Odometry dataset [39], where the objective is to track a vehicle's pose over time using RGB images captured from a stereo camera and the given initial pose. In this task, the vehicle's pose is not directly available from images and must be inferred by estimating relative motion across frames and accumulating it over time. This process is prone to drift, especially without loop closure. Moreover, the limited size and diversity of the dataset pose challenges for generalization to unseen road conditions.

Data: Following the protocol in [14], [29], we evaluate DiffPF on the KITTI-10 dataset, which consists of ten urban

TABLE VI: Performance comparison of DiffPF and baseline models on KITTI dataset.

Setting	Methods	m/m	deg/m)
Ours	DiffPF	0.126 ± 0.019	0.0549 ± 0.005
Differentiable Filter	dUKF	0.180 ± 0.023	0.0799 ± 0.008
	dEKF	0.168 ± 0.012	0.0743 ± 0.007
	dPF-M-lrn	0.190 ± 0.030	0.0900 ± 0.014
	dMCKF	0.200 ± 0.030	0.0820 ± 0.013
LSTM	LSTM (uni.)	0.757 ± 0.012	0.3779 ± 0.054
	LSTM (dy.)	0.538 ± 0.057	0.0788 ± 0.008
Smoother,	E2E Loss, Vel	0.148 ± 0.009	0.0720 ± 0.006
Heteroscedastic	E2E Loss, Pos	0.146 ± 0.010	0.0762 ± 0.007

driving sequences with paired stereo images and ground-truth poses recorded at approximately 10 Hz. Stereo image pairs are horizontally flipped for data augmentation, and the sequences are segmented into fixed-length clips and downsampled to 50×150 pixels.

Implementation: The vehicle state is represented as a 5D vector and observations include the current RGB frame and a difference frame computed from the previous frame [14], [40]. The process model design, sensor model network, and denoising network follow those in [29]. The predicted particle set is first flattened and concatenated with the observation features to form the conditioning input for the diffusion model. Training follows a 10-fold cross-validation protocol, with the model pretrained using only visual observations before introducing the particle set.

Results: We evaluate on 100-frame segments following KITTI’s standard metrics. Positional and angular errors are computed by comparing the final predicted pose to the ground truth and normalizing by the total traveled distance (m/m and deg/m, respectively).

To ensure a more thorough evaluation, a diverse set of baseline models is selected for comparison:

1) Differentiable Filters: Include several state-of-the-art differentiable recursive filters proposed in previous studies [14], [29], [40]. To ensure a fair evaluation, we adopt the same input and use identical sensor model architectures.

2) LSTMs: Building on previous research in differentiable filtering [14], [40], we include long short-term memory (LSTM) models [41]. We choose an unidirectional LSTM and a LSTM integrated with dynamics as the baselines [14].

3) Smoothers: We adopt the differentiable smoothers introduced in [35], which are built upon factor graph optimization. As these methods utilize the full observation sequence, they naturally hold an advantage over filtering-based approaches.

All of the experimental results are presented in Table VI.

Comparison: Our DiffPF achieves a 25% reduction in position error and a 26% reduction in orientation error compared to leading differentiable filtering approaches. It also surpasses LSTMs and differentiable smoothers in performance. These results highlight the strength of the diffusion model in capturing complex posterior distributions, enabling more effective particle sampling and higher-quality approximations of the posterior. Furthermore, the method operates at 46.5 Hz with a memory footprint of 222.7 MB, well above the 10 Hz input

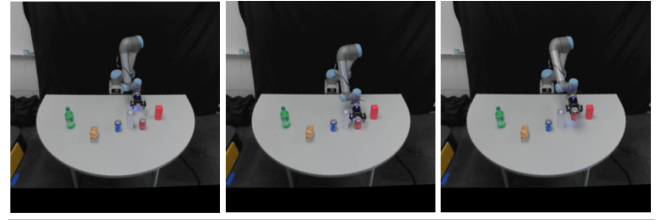


Fig. 7: Example image sequence from the UR5 manipulation task

TABLE VII: Result evaluations on UR5 manipulation task

Method	Real-world (MAE)		Simulation (MAE)	
	Joint (deg)	EE (cm)	Joint (deg)	EE (cm)
dEKF [14]	16.08±0.10	5.67±0.10	4.93±0.20	1.91±0.10
DPF [6]	15.93±0.10	5.08±0.30	4.46±0.20	1.51±0.20
dPF-M [14]	12.83±0.10	3.95±0.40	3.82±0.20	1.26±0.10
α -MDF [36]	7.49±0.10	3.81±0.20	2.84±0.10	1.06±0.10
DEnKF [37]	11.42±0.01	3.42±0.01	2.56±0.09	0.82±0.02
DiffPF	4.27±0.58	1.77±0.23	0.50±0.02	0.27±0.01

Means ± standard errors.

rate of the visual stream, ensuring its suitability for real-time applications.

D. Robotic Manipulation

Overview: We evaluate our method on a robot manipulation state estimation task described in [36]. The objective is to track the internal state of a UR5 robot during tabletop manipulation from RGB image sequences, as illustrated in Fig. 7. Under partial observability, the system must estimate and propagate a higher-dimensional robot state purely from visual inputs. Experiments are conducted in both simulated and real-world environments to assess robustness across domains.

Data: Following the protocol and dataset of [36], we consider UR5 manipulation sequences collected in simulation and on a real robot. The internal state $x \in \mathbb{R}^{10}$ composed of 7 joint angles and the Cartesian position of the end-effector. RGB images are used as observation. All sequences are segmented into fixed-length clips for training and evaluation.

Implementation: Since actions are not available and the data are sampled at a relatively high frequency, the process model adopts an identity transition, setting $\hat{x}_t^{(i)} = x_{t-1}^{(i)}$. The sensor model network follows the design in [36]. To stabilize training, we first pretrain the model using visual observations only.

Results: We evaluate performance using mean absolute error (MAE) on joint angles and end-effector positions. All results are reported in comparison with the methods and baselines presented in [36], [37] and presented in Table VII.

Comparison: DiffPF achieves substantially lower estimation errors than existing differentiable filtering methods on the UR5 manipulation task. In real-world experiments, it reduces end-effector position error by 48% and joint angle error by 43% compared to the strongest baseline, while exhibiting consistent improvements in simulation. In addition, DiffPF operates at over 50 Hz, demonstrating its real-time applicability. These results further indicate that DiffPF maintains strong performance even when scaling to higher-dimensional state spaces.

V. CONCLUSIONS

In this work, we introduced **DiffPF**, a novel differentiable particle filtering framework that integrates diffusion models into the recursive Bayesian estimation pipeline. DiffPF replaces hand-designed proposals and importance weighting with a learned denoising process, enabling flexible and fully differentiable sampling from complex posteriors. This conditional diffusion formulation effectively captures multimodal distributions, mitigates particle degeneracy, and removes the need for resampling, yielding more expressive and robust belief representations. Experiments on both simulated and real-world tasks demonstrate consistent performance gains over strong baselines, highlighting the generality and robustness of our approach. Future work may focus on the potential risk of sample collapse and mode dropping by exploring weighted variants of DiffPF, where importance sampling could further improve robustness.

REFERENCES

- [1] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics (Intelligent Robotics and Autonomous Agents)*. The MIT Press, 2005.
- [2] N. J. Gordon, D. J. Salmond, and A. F. Smith, "Novel approach to nonlinear/non-gaussian bayesian state estimation," in *IEE proceedings F (radar and signal processing)*, vol. 140, no. 2. IET, 1993, pp. 107–113.
- [3] K. Murphy and S. Russell, "Rao-blackwellised particle filtering for dynamic bayesian networks," in *Sequential Monte Carlo methods in practice*. Springer, 2001, pp. 499–515.
- [4] S. S. Rangapuram, M. W. Seeger, J. Gasthaus, L. Stella, Y. Wang, and T. Januschowski, "Deep state space models for time series forecasting," *Advances in neural information processing systems*, vol. 31, 2018.
- [5] X. Chen and Y. Li, "An overview of differentiable particle filters for data-adaptive sequential bayesian inference," *arXiv preprint arXiv:2302.09639*, 2023.
- [6] R. Jonschkowski, D. Rastogi, and O. Brock, "Differentiable particle filters: End-to-end learning with algorithmic priors," *arXiv preprint arXiv:1805.11122*, 2018.
- [7] P. Karkus, D. Hsu, and W. S. Lee, "Particle filter networks with application to visual localization," in *Conference on robot learning*. PMLR, 2018, pp. 169–178.
- [8] P. M. Djuric, J. H. Kotecha, J. Zhang, Y. Huang, T. Ghirmai, M. F. Bugallo, and J. Miguez, "Particle filtering," *IEEE signal processing magazine*, vol. 20, no. 5, pp. 19–38, 2003.
- [9] S. Godsill, "Particle filtering: the first 25 years and beyond," in *ICASSP 2019-2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2019, pp. 7760–7764.
- [10] X. Chen and Y. Li, "Normalizing flow-based differentiable particle filters," *IEEE Transactions on Signal Processing*, 2024.
- [11] A. Corenflos, J. Thornton, G. Deligiannidis, and A. Doucet, "Differentiable particle filtering via entropy-regularized optimal transport," in *International Conference on Machine Learning*. PMLR, 2021, pp. 2100–2111.
- [12] A. Ścibior and F. Wood, "Differentiable particle filtering without modifying the forward pass," *arXiv preprint arXiv:2106.10314*, 2021.
- [13] C. Rosato, L. Devlin, V. Beraud, P. Horridge, T. B. Schön, and S. Maskell, "Efficient learning of the parameters of non-linear models using differentiable resampling in particle filters," *IEEE Transactions on Signal Processing*, vol. 70, pp. 3676–3692, 2022.
- [14] A. Kloss, G. Martius, and J. Bohg, "How to train your differentiable filter," *Autonomous Robots*, vol. 45, no. 4, pp. 561–578, 2021.
- [15] X. Ma, P. Karkus, D. Hsu, and W. S. Lee, "Particle filter recurrent neural networks," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 04, 2020, pp. 5101–5108.
- [16] T. Le, M. Igl, T. Rainforth, T. Jin, and F. Wood, "Auto-encoding sequential monte carlo," in *International Conference on Learning Representations (ICLR)*. OpenReview, 2018.
- [17] C. Naesseth, S. Linderman, R. Ranganath, and D. Blei, "Variational sequential monte carlo," in *International conference on artificial intelligence and statistics*. PMLR, 2018, pp. 968–977.
- [18] P. Bickel, B. Li, and T. Bengtsson, "Sharp failure rates for the bootstrap particle filter in high dimensions," in *Pushing the limits of contemporary statistics: Contributions in honor of Jayanta K. Ghosh*. Institute of Mathematical Statistics, 2008, vol. 3, pp. 318–330.
- [19] M. Zhu, K. Murphy, and R. Jonschkowski, "Towards differentiable resampling," *arXiv preprint arXiv:2004.11938*, 2020.
- [20] S. Liu, W. Cao, C. Liu, Z. He, T. Zhang, and S. E. Li, "One filters all: A generalist filter for state estimation," 2025. [Online]. Available: <https://arxiv.org/abs/2509.20051>
- [21] S. K. Lind, J. Li, M. Stenmark, and V. Krüger, "Normalizing flows are capable visuomotor policy learning models," 2025. [Online]. Available: <https://arxiv.org/abs/2509.21073>
- [22] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, and S. Ganguli, "Deep unsupervised learning using nonequilibrium thermodynamics," in *International conference on machine learning*. PMLR, 2015, pp. 2256–2265.
- [23] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," *Advances in neural information processing systems*, vol. 33, pp. 6840–6851, 2020.
- [24] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," *The International Journal of Robotics Research*, p. 02783649241273668, 2023.
- [25] A. Sridhar, D. Shah, C. Glossop, and S. Levine, "Nomad: Goal masked diffusion policies for navigation and exploration," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 63–70.
- [26] Z. Wang, J. J. Hunt, and M. Zhou, "Diffusion policies as an expressive policy class for offline reinforcement learning," *arXiv preprint arXiv:2208.06193*, 2022.
- [27] J. Carvalho, A. T. Le, M. Baierl, D. Koert, and J. Peters, "Motion planning diffusion: Learning and planning of robot motions with diffusion models," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2023, pp. 1916–1923.
- [28] Z. Liang, Y. Mu, M. Ding, F. Ni, M. Tomizuka, and P. Luo, "Adapt-diffuser: Diffusion models as adaptive self-evolving planners," *arXiv preprint arXiv:2302.01877*, 2023.
- [29] Z. Wan and L. Zhao, "Dnd filter: Differentiable state estimation for dynamic systems using diffusion models," *arXiv preprint arXiv:2503.01274*, 2025.
- [30] S. Särkkä and L. Svensson, *Bayesian filtering and smoothing*. Cambridge university press, 2023, vol. 17.
- [31] L. V. Jospin, H. Laga, F. Boussaid, W. Buntine, and M. Bennamoun, "Hands-on bayesian neural networks—a tutorial for deep learning users," *IEEE Computational Intelligence Magazine*, vol. 17, no. 2, pp. 29–48, 2022.
- [32] G. Papamakarios, E. Nalisnick, D. J. Rezende, S. Mohamed, and B. Lakshminarayanan, "Normalizing flows for probabilistic modeling and inference," *Journal of Machine Learning Research*, vol. 22, no. 57, pp. 1–64, 2021.
- [33] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," *arXiv preprint arXiv:2010.02502*, 2020.
- [34] S. Chan et al., "Tutorial on diffusion models for imaging and vision," *Foundations and Trends® in Computer Graphics and Vision*, vol. 16, no. 4, pp. 322–471, 2024.
- [35] B. Yi, M. A. Lee, A. Kloss, R. Martín-Martín, and J. Bohg, "Differentiable factor graph optimization for learning smoothers," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2021, pp. 1339–1345.
- [36] X. Liu, Y. Zhou, S. Ikemoto, and H. B. Amor, "α-mdf: An attention-based multimodal differentiable filter for robot state estimation," in *7th Annual Conference on Robot Learning*, 2023.
- [37] X. Liu, G. Clark, J. Campbell, Y. Zhou, and H. B. Amor, "Enhancing state estimation in robots: A data-driven approach with differentiable ensemble kalman filters," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2023, pp. 1947–1954.
- [38] C. Beattie, J. Z. Leibo, D. Teplyashin, T. Ward, M. Wainwright, H. Küttler, A. Lefrancq, S. Green, V. Valdés, A. Sadik, et al., "Deepmind lab," *arXiv preprint arXiv:1612.03801*, 2016.
- [39] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, "Vision meets robotics: The kitti dataset," *The International Journal of Robotics Research*, vol. 32, no. 11, pp. 1231–1237, 2013.
- [40] T. Haarnoja, A. Ajay, S. Levine, and P. Abbeel, "Backprop kf: Learning discriminative deterministic state estimators," *Advances in neural information processing systems*, vol. 29, 2016.
- [41] A. Graves and A. Graves, "Long short-term memory," *Supervised sequence labelling with recurrent neural networks*, pp. 37–45, 2012.